```
            return name;
        }

        @Override
        public int hashCode() {
            return Arrays.hashCode(this.distribution)+this.name.hashCode();
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) {
                return true;
            }
            if (obj == null) {
                return false;
            }
            if (getClass() != obj.getClass()) {
                return false;
            }
            final DicePlayer other = (DicePlayer) obj;
            if (!Objects.equals(this.name, other.name)) {
                return false;
            }
            return Arrays.equals(this.distribution, other.distribution);
        }
    }
```

***Κώδικας 11.17*** *Η κλάση DicePlayer*

Ο δημιουργός της κλάσης DicePlayer του κώδικα 11.17 παρέχει τη δυνατότητα να κατασκευάζονται παίκτες με διαφορετικά χαρακτηριστικά ζαριάς. Λαμβάνει 2 παραμέτρους, μια για το όνομα του παίκτη και μια για την κατανομή που καθορίζει τα αποτελέσματα των ρίψεων ζαριού για τον συγκεκριμένο παίκτη. Ο δημιουργός ελέγχει ώστε το σύνολο των πιθανοτήτων για να φέρει ένα αποτέλεσμα από 1 έως 6 ο παίκτης να είναι ίσο με 1. Επίσης, παρέχεται ένας δημιουργός με παράμετρο μόνο το όνομα του παίκτη που κατασκευάζει έναν παίκτη με ίση πιθανότητα για κάθε πιθανή ζαριά.

Προσέξτε πως τα μέλη name και distribution είναι final ενώ δεν παρέχονται για αυτά συναρτήσεις ρυθμιστές. Έτσι μετά τη δημιουργία ενός παίκτη δεν υπάρχει η δυνατότητα να αλλάξει το όνομά του ή τα χαρακτηριστικά της ζαριάς του.

Η συνάρτηση roll βασίζεται στον αλγόριθμο που περιγράφεται στον κώδικα 8.4.

Οι equals και hashCode είναι σχετικά απλές, κρίσιμες όμως στην συγκεκριμένη εφαρμογή, καθώς όπως θα δούμε στην DiceGame χρησιμοποιείται ένα HashMap που τις αξιοποιεί.

```
import java.util.HashMap;

public class DiceGame {

    private final int noOfPlayers;
    private final int noOfRoll;

    private final DicePlayer john = new DicePlayer("John");
    private final DicePlayer jim = new DicePlayer("Jim", new double[]{1d /
6, 1d / 6, 1d / 6, 1d / 6, 0, 2d / 6});
    private final DicePlayer jack = new DicePlayer("Jack", new double[]{1d /
6, 1d / 6, 1d / 6, 1d / 6, 2d / 6, 0});

    private final HashMap<DicePlayer, Integer> scores = new HashMap<>();

    public DiceGame(int noOfPlayers, int noOfRoll) {
        this.noOfPlayers = noOfPlayers;
        this.noOfRoll = noOfRoll;
```

```
}

private void updScores(DicePlayer player) {
    Integer score = scores.get(player);
    if (score == null) {
        score = 0;
    }
    scores.put(player, score + 1);
}

private void aLot() {

    for (int i = 0; i < noOfRoll; i++) {
        int rslt = john.roll();
        if (rslt % 2 == 0) {
            updScores(john);
        }

        rslt = jim.roll();
        if (rslt % 2 == 0) {
            updScores(jim);
        }

        rslt = jack.roll();
        if (rslt % 2 == 0) {
            updScores(jack);
        }
    }
}

private boolean withdraw() {
    int score1 = scores.get(john),
            score2 = scores.get(jim),
            score3 = scores.get(jack);
    return (score1 == score2 || score2 == score3 || score1 == score3);
}

private DicePlayer max() {
    int scoreJohn = scores.get(john) != null ? scores.get(john) : 0;
    int scoreJim = scores.get(jim) != null ? scores.get(jim) : 0;
    int scoreJack = scores.get(jack) != null ? scores.get(jack) : 0;
    int max = scoreJohn;
    DicePlayer rVal = john;
    if (scoreJim > max) {
        max = scoreJim;
        rVal = jim;
    }
    if (scoreJack > max) {
        rVal = jack;
    }
    return rVal;
}

public DicePlayer play() {
    do {
        aLot();
    } while (withdraw());
    return max();
}

public int getNoOfPlayers() {
```

```
        return noOfPlayers;
    }

    public int getNoOfRoll() {
        return noOfRoll;
    }

    public static void main(String[] args) {
        DiceGame diceGame = new DiceGame(3, 10);
        DicePlayer winner = diceGame.play();
        System.out.println("Winner is " + winner.getName() + " Total Score "
+ diceGame.scores.get(winner));
    }
}
```

***Κώδικας 11.18*** *Η κλάση DiceGame*

Η μέθοδος play στον κώδικα 11.18 επαναλαμβάνει μια παρτίδα ζαριών ενόσω εντοπίζεται ισοπαλία. Με τη λήξη της ισοπαλίας επιστρέφει τον παίκτη που πέτυχε το μεγαλύτερο σκορ. Στα ιδιωτικά μέλη της κλάσης περιλαμβάνεται το μέλος scores τύπου HashMap<DicePlayer, Integer>. Πρόκειται για μια κλάση που ανήκει στο πακέτο java.util και πιο συγκεκριμένα για μία γενίκευση. Αναλυτικά θα μιλήσουμε για τις γενικεύσεις στην ενότητα 17. Για την ώρα μας αρκεί να γνωρίζουμε πως χρησιμοποιούμε το scores στα πλαίσια της άσκησής μας. Το scores αποτελεί ένα ευρετήριο του οποίου το κλειδί είναι τύπου DicePlayer και η τιμή τύπου Integer. Με άλλα λόγια, συσχετίζει έναν παίκτη με μια ακέραια τιμή που αντιπροσωπεύει το σκορ του παίκτη. Παράλληλα, έχουν δηλωθεί και αρχικοποιηθεί και οι τρεις παίκτες της άσκησης.

Η συνάρτηση aLot προσομοιώνει μια παρτίδα. Μέσα σε έναν βρόγχο 10 επαναλήψεων, κάθε παίκτης ρίχνει μια ζαριά. Αν το αποτέλεσμα είναι άρτιο καλεί την updScores και ενημερώνει το σκορ του παίκτη. Η updateScores με την σειρά της ελέγχει αν στο scores υπάρχει το score του παίκτη. Αν δεν βρεθεί η get επιστρέφει null. Σε αυτήν την περίπτωση αρχικοποιείται το σκορ του παίκτη και ενημερώνεται η scores. Σημειώστε πως η put αν δεν βρεθεί εγγραφή με αυτόν τον παίκτη εισάγει νέα εγγραφή, ενώ αν βρεθεί, ενημερώνει την υπάρχουσα.

Η withdraw επιστρέφει true αν εντοπίσει έστω και ένα ζεύγος ισοβαθμούντων παικτών. Τέλος η max επιστρέφει τον παίκτη με το μεγαλύτερο σκορ από τους τρεις.

Η άσκηση μπορεί να λυθεί και χωρίς την HashMap, ωστόσο είναι χρήσιμο να ανακαλύπτουμε νέες κλάσεις από τη βιβλιοθήκη της Java.

### 11.9.7 Η Τράπουλα

Μια τράπουλα αποτελείται από 52 κάρτες, δηλαδή από όλες τις τυπικές κάρτες χωρίς μπαλαντέρ. Κάθε κάρτα έχει δύο χαρακτηριστικά, το χρώμα και τον βαθμό. Το χρώμα λαμβάνει τις τιμές καρό, κούπα, σπαθί και μπαστούνι. Ο δε βαθμός λαμβάνει τις τιμές 2,3,…, Βαλές, Ντάμα, Ρήγας, Άσος.

Να υλοποιηθεί η κλάση Deck (Τράπουλα). Το χρώμα και ο βαθμός κάθε κάρτας να κωδικοποιηθούν ως απαριθμήσιμοι τύποι.

Να υλοποιηθεί εφαρμογή που δημιουργεί δύο τράπουλες. Ανοίγει την πρώτη, δηλαδή εμφανίζει όλες τις κάρτες της, τέσσερις ανά γραμμή με την ακόλουθη σειρά: Οι κάρτες ενός βαθμού εμφανίζονται στην ίδια γραμμή πρώτα οι κούπες, μετά τα μπαστούνια ακολουθούμενα από τα σπαθιά και τέλος τα καρό, όπως δείχνει το παρακάτω απόσπασμα εξόδου.

```
TWO HEARTS TWO SPADES TWO CLUBS TWO DIAMONDS
THREE HEARTS THREE SPADES THREE CLUBS THREE DIAMONDS
…
…
TEN HEARTS TEN SPADES TEN CLUBS TEN DIAMONDS
JACK HEARTS JACK SPADES JACK CLUBS JACK DIAMONDS
QUEEN HEARTS QUEEN SPADES QUEEN CLUBS QUEEN DIAMONDS
KING HEARTS KING SPADES KING CLUBS KING DIAMONDS
ACE HEARTS ACE SPADES ACE CLUBS ACE DIAMONDS
```

Στη συνέχεια, ανακατέψτε τη δεύτερη τράπουλα και ανοίξτε τις κάρτες της.

*Υπόδειξη*: Προγραμματίστε κατάλληλα τη μέθοδο toString ώστε να εμφανίζει τις κάρτες της τράπουλας, τέσσερις ανά γραμμή με τη σειρά που είναι τοποθετημένες οι κάρτες στην τράπουλα.

**Λύση**
Έναν απαριθμήσιμο τύπο για να αναπαραστήσουμε τον βαθμό μιας κάρτας τράπουλας έχουμε δώσει στην ενότητα 11.6. Πρόκειται για τον τύπο CardRank. Ο τύπος είναι κατάλληλος για τις απαιτήσεις της άσκησης, οπότε θα χρησιμοποιήσουμε αυτόν. Ένας κατάλληλος τύπος για την αναπαράσταση του χρώματος δίνεται στον κώδικα 11.19:

```
public enum Suit { //Κώδικας 11.19
  HEARTS,
  SPADES,
  CLUBS,
  DIAMONDS;
}
```

***Κώδικας 11.19*** *Αναπαράσταση του χρώματος κάρτας τράπουλας*

Στη συνέχεια, η κλάση που αναπαριστά κάρτες μπορεί να αναπτυχθεί όπως δείχνει ο κώδικας 11.20:

```
public class Card {

    private final Suit suit;
    private final CardRank cardRank;

    public Card(CardRank cardRank, Suit suit) {
        this.cardRank = cardRank;
        this.suit = suit;
    }

    public Suit getSuit() {
        return suit;
    }

    public CardRank getCardRank() {
        return cardRank;
    }

    @Override
    public String toString() {
        return this.cardRank + " " + this.suit;
    }
}
```

***Κώδικας 11.20*** *Κλάση αναπαράστασης κάρτας τράπουλας*

Ας σημειωθεί πως σε αυτήν τη λύση, για οικονομία χώρου, συμπεριλαμβάνουμε μόνο συναρτήσεις που είναι αναγκαίες για τη συγκεκριμένη εφαρμογή. Μπορείτε να συμπληρώσετε τις υπόλοιπες αναγκαίες συναρτήσεις στις κλάσεις μόνοι σας για εξάσκηση.

Όσον αφορά την κλάση Card, είναι μια πολύ απλή κλάση που συνδυάζει ένα αντικείμενο Suite και ένα CardRank. Διαθέτει αναγνώστες αλλά όχι ρυθμιστές, καθώς τόσο η cardRank όσο και η suite είναι final. Αυτή η επιλογή υπαγορεύεται από την ανάγκη να μην αλλάζει ο βαθμός ή το χρώμα μιας κάρτας μετά τη δημιουργία της, κάτι που ισχύει και στην πραγματικότητα του φυσικού κόσμου.

Τώρα μπορούμε να δούμε και την κλάση που αναπαριστά τράπουλες:

```
import java.util.Random;
```

```
public class Deck {

    private final Card[] deck;
    private int cardsInDeck;

    public Deck() {
        deck = new Card[52];
        cardsInDeck = 0;
        for (CardRank r : CardRank.values()) {
            for (Suit s : Suit.values()) {
                deck[cardsInDeck++] = new Card(r, s);
            }
        }
    }

    public void shuffle() {
        Random r = new Random();

        for (int i = 0; i < deck.length; i++) {
            int rIdx = r.nextInt(deck.length);
            Card tmp = deck[i];
            deck[i] = deck[rIdx];
            deck[rIdx] = tmp;
        }
    }

    public boolean empty() {
        return cardsInDeck == 0;
    }

    public Card pickCard() {
        cardsInDeck--;
        return deck[52 - cardsInDeck - 1];
    }

    @Override
    public String toString() {
        if (cardsInDeck == 0) {
            return "Empty Deck";
        }
        String rVal = "";
        for (int i = 52 - cardsInDeck; i < 52; i++) {
            if (i > 52 - cardsInDeck && i % 4 == 0) {
                rVal += "\n";
            }
            rVal += deck[i] + " ";
        }
        return rVal;
    }

    public static void main(String[] args) {
        Deck d1 = new Deck();
        Deck d2 = new Deck();

        for (int i = 0; i < 24; i++) {
            d1.pickCard();
        }
        System.out.println(d1.cardsInDeck);
        System.out.println(d1);
        System.out.println("");
```

```
            d2.shuffle();
            System.out.println(d2);
        }
    }
```

***Κώδικας 11.21*** *Η κλάση Deck*

Όπως φαίνεται στον κώδικα 11.21, η κλάση Deck αποθηκεύει τις κάρτες στον πίνακα-ιδιωτικό μέλος deck. Επίσης, διαθέτει το ιδιωτικό μέλος cardsInDeck το οποίο παρακολουθεί πόσες κάρτες εναπομένουν στην τράπουλα. Τόσο η μεταβλητή deck όσο και η cardsInDeck λαμβάνουν τιμή στον δημιουργό της κλάσης. Εκεί δημιουργούνται οι κάρτες που εισάγονται στην τράπουλα με τη σειρά που μας ζητάει η εκφώνηση να εμφανιστούν οι κάρτες μιας μη ανακατεμένης τράπουλας. Με την ολοκλήρωση της εκτέλεσης του δημιουργού, η τράπουλα διαθέτει και τα 52 φύλλα στην απαιτούμενη σειρά και η μεταβλητή cardsInDeck έχει την τιμή 52. Η συνάρτηση isEmpty απλά επιστρέφει true όταν η μεταβλητή cardsInDeck είναι ίση με το 0. Η cardsInDeck μειώνεται από τη μέθοδο pickCard η οποία επιστρέφει μια κάρτα από την τράπουλα. Ποια κάρτα όμως ακριβώς επιστρέφει δεδομένου ότι όλα τα αντικείμενα τύπου Card παραμένουν ως στοιχεία του πίνακα deck;

Όπως φαίνεται στον κώδικα 11.21, η pickCard σηκώνει την κάρτα στη θέση 52 - cardsInDeck – 1 του deck. Πράγματι, αν υποθέσουμε πως η cardsInDeck έχει την τιμή 28, δηλαδή έχουν ληφθεί 24 κάρτες (μέσω της pickCard) από την τράπουλα των 52 φύλλων, σε ποια θέση βρίσκεται η επόμενη διαθέσιμη κάρτα; Εφόσον, έχουν ληφθεί 24 κάρτες από την τράπουλα, η επόμενη διαθέσιμη κάρτα βρίσκεται στη θέση 25, δηλαδή στη θέση 52 μείον 28 που είναι η τιμή της cardsInDeck μείον 1 λόγω του ότι ο πίνακας είναι zero based.

Η μέθοδος toString ελέγχει πρώτα αν η τράπουλα είναι άδεια οπότε επιστρέφει τη σειρά “Empty Deck”. Αν δεν επιστρέψει “Empty Deck”, συνεχίζει και στήνει στην rVal τις κάρτες της τράπουλας. Πρέπει ωστόσο η rVal να συμπεριλαμβάνει μόνο τις κάρτες που έχουν απομείνει. Αν έχουν για παράδειγμα απομείνει 5 κάρτες, δηλαδή cardsInDeck==5, δεδομένου ότι η pickCard επιστρέφει την πρώτη εναπομένουσα κάρτα, αυτές είναι οι 5 κάρτες που είναι τοποθετημένες τελευταίες στον πίνακα deck. Επομένως, η πρόσθεση των καρτών στην rVal θα πρέπει να αρχίσει από τη θέση 52-cardsInDeck του deck.

Δεδομένου ότι η εκφώνηση μας ζητάει να εμφανίζονται τέσσερις κάρτες από έναν βαθμό (εννοείται εφόσον υπάρχουν και δεν έχουν ληφθεί ήδη από την pickCard) σε μια γραμμή, ελέγχουμε και εφόσον η μεταβλητή i διαιρείται ακριβώς με το τέσσερα και είναι μεγαλύτερη από την τιμή 52-cardsInDeck, δηλαδή δεν πρόκειται για την πρώτη γραμμή, τότε προσθέτουμε τον χαρακτήρα new line στην rVal. Στο τέλος της επαναληπτικής διαδικασίας, η rVal έχει το απαιτούμενο περιεχόμενο οπότε την επιστρέφουμε.

Τέλος, έχοντας στη διάθεσή μας την κλάση Deck και τις λοιπές βοηθητικές κλάσεις, η εφαρμογή που ζητά η εκφώνηση είναι απλή και παρουσιάζεται στον κώδικα 11.22.

```
public class App {

    public static void main(String[] args) {
        Deck d1 = new Deck();
        Deck d2 = new Deck();
        System.out.println(d1);
        d2.shuffle();
        System.out.println("");
        System.out.println(d2);
    }
}
```

***Κώδικας 11.22*** *Η εφαρμογή της άσκησης 11.9.7*

## 11.10 Ασκήσεις προς λύση

### 11.10.1 Η κλάση των Πρώτων Αριθμών

Να υλοποιηθεί η κλάση PrimeNumber. Κάθε αντικείμενο της κλάσης αναπαριστά έναν πρώτο αριθμό. Η κλάση διαθέτει τα ακόλουθα μέλη:

```
PrimeNumber(int val)
```

Ο δημιουργός κάνει έλεγχο ώστε το αντικείμενο να αντιπροσωπεύει πράγματι έναν πρώτο αριθμό.

```
public static boolean isPrime(int val)
```
Ελέγχει αν η ακέραιη παράμετρός της είναι πρώτος αριθμός.

```
public static int getNext(int val)
```
Επιστρέφει τον μικρότερο αριθμό που είναι πρώτος και μεγαλύτερος από την ακέραια παράμετρό της.

```
PrimeNumber getNext()
```
Επιστρέφει τον μικρότερο αριθμό που είναι πρώτος και μεγαλύτερος από το αντικείμενο που την καλεί.
Να προστεθούν όλες οι αναγκαίες μέθοδοι. Να προστεθεί main που εμφανίζει τους 100 πρώτους αριθμούς.

### 11.10.2 Η κλάση SetOfStrings

Η κλάση SetOfStrings αναπαριστά σύνολα από Strings. Σε ένα σύνολο δεν μπορεί να υπάρχει 2 φορές το ίδιο στοιχείο. Η κλάση περιλαμβάνει τα ακόλουθα μέλη:

```
public SetOfStrings(int size)
```
Δημιουργεί ένα αντικείμενο με αρχικό αποθηκευτικό χώρο size.

```
public SetOfStrings()
```
Δημιουργεί ένα σύνολο με αρχικό αποθηκευτικό χώρο ίσο με 10.

```
public SetOfStrings(SetOfStrings source)
```
Δημιουργεί ένα αντίγραφο του source

```
public boolean contains(String s)
```
Ελέγχει αν αυτό το σύνολο περιλαμβάνει την αλφαριθμητική σειρά s

```
public void add(String s)
```
Προσθέτει την αλφαριθμητική σειρά s σε αυτό το σύνολο. Αν η s υπάρχει ήδη, η add τερματίζει χωρίς να μεταβάλει αυτό το σύνολο. Αν η s δεν υπάρχει ήδη, ελέγχεται η χωρητικότητα, αν επαρκεί προστίθεται το s αλλιώς αυξάνεται η χωρητικότητα και στη συνέχεια προστίθεται το s.

```
public boolean remove(String s)
```
Διαγράφει τη σειρά s από αυτό το σύνολο. Επιστρέφει true αν η σειρά s βρέθηκε και διαγράφηκε από αυτό το σύνολο και false αν η σειρά δεν βρέθηκε.

```
public int size()
```
Επιστρέφει το μέγεθος αυτού του συνόλου, δηλαδή τον αριθμό των στοιχείων του

```
public String get(int i)
```
Επιστρέφει το στοιχείο που βρίσκεται στη θέση i αυτού του συνόλου

```
public SetOfStrings union(SetOfStrings sS)
```
Επιστρέφει την ένωση αυτού του συνόλου με το σύνολο sS

```
public SetOfStrings intersection(SetOfStrings sS)
```
Επιστρέφει την τομή αυτού του συνόλου με το sS

Να υλοποιηθεί η κλάση. Να προστεθούν όλες οι αναγκαίες μέθοδοι. Να προστεθεί main στην οποία να δημιουργείται ένα SetOfStrings με στοιχεία “0”,”1”,”2”,…,”30” και ένα δεύτερο με στοιχεία “20”,”21”,”22”,…,”50”. Να εμφανιστεί η ένωση και η τομή των δύο συνόλων. Χρησιμοποιήστε για αποθηκευτικό χώρο πίνακα από Strings. Δημιουργήστε τον για ένα αρχικό μήκος. Αναπροσαρμόστε το μήκος αυτόματα όποτε ο χώρος δεν επαρκεί.

### 11.10.3 Η κλάση StackOfStrings

Η κλάση StackOfStrings αναπαριστά μία στοίβα από Strings. Ένα stack, όπως εξηγείται στην ενότητα 9.1, είναι μια δομή LIFO (Last In First Out). Η κλάση περιλαμβάνει τις ακόλουθες μεθόδους:

```
public boolean isEmpty()
```

Επιστρέφει true αν αυτό το stack είναι άδειο.

```
public void push(String data)
```

Εισάγει στην κορυφή του stack το String data.

```
public String pop()
```

Αφαιρεί το στοιχείο από την κορυφή του stack και το επιστρέφει.

Χρησιμοποιείστε για αποθηκευτικό χώρο πίνακα από Strings. Δημιουργήστε τον για ένα αρχικό μήκος. Αναπροσαρμόστε το μήκος αυτόματα, όποτε ο χώρος δεν επαρκεί.

### 11.10.4 Η κλάση QuadraticEquation

Να υλοποιηθεί η κλάση QuadraticEquation. Κάθε αντικείμενο της κλάσης αντιπροσωπεύει μια εξίσωση β' βαθμού. Η κλάση περιλαμβάνει τις ακόλουθες μεθόδους:

```
public QuadraticEquation (double a, double b, double g)
```

Δημιουργεί ένα αντικείμενο.

```
public QuadraticEquation (double[] triwnymo)
```

Δημιουργεί ένα αντικείμενο.

```
public double discriminant()
```

Επιστρέφει τη διακρίνουσα αυτής της εξίσωσης.

```
public double x1()
```

Επιστρέφει τη μια από τις ρίζες αυτής της εξίσωσης. Αν η διακρίνουσά της είναι μικρότερη από το 0, επιστρέφει Double.NaN.

```
public double x2()
```

Επιστρέφει τη μια από τις ρίζες αυτής της εξίσωσης. Αν η διακρίνουσά της είναι μικρότερη από το 0, επιστρέφει Double.NaN. Αν η διακρίνουσα είναι ίση με το 0, τότε οι δύο ρίζες είναι ίσες.

### 11.10.5 Η κλάση MyDate

Να υλοποιηθεί η κλάση MyDate που περιλαμβάνει τρεις μεταβλητές στιγμιότυπου ως εξής:

```
private int day;
private MyMonth month;
private int year;
```

Η day αναπαριστά την ημέρα ενός μήνα, η year το έτος και η month τον μήνα. Ενώ η day και η year είναι ακέραιοι, η month είναι απαριθμήσιμος τύπος.

Η κλάση διαθέτει επίσης ένα στατικό πεδίο format το οποίο είναι επίσης απαριθμήσιμος τύπος με δύο αντικείμενα, το GREEK και το ENGLISH που καθορίζουν την επιστροφή της toString ως εξής: Αν η format έχει την τιμή GREEK τότε η ημερομηνία επιστρέφεται ως dd/mm/yy διαφορετικά ως mm/dd/yy.

Η κλάση διαθέτει μια στατική μέθοδο isDisekto(int) που επιστρέφει true εφόσον καλείται με παράμετρο που αντιπροσωπεύει δίσεκτο έτος. Σημειώστε πως δίσεκτα είναι τα έτη που διαιρούνται ακριβώς με το 400 ή διαιρούνται με το 4 αλλά όχι με το 100. Επιπλέον, η κλάση διαθέτει μια μέθοδο στιγμιότυπου isDisekto που επιστρέφει true εφόσον κληθεί από αντικείμενο δίσεκτου έτους.

Προσθέστε main που χαρακτηρίζει κάθε έτος από το 2017 έως το 2100 ως δίσεκτο ή μη.

Πρόχειρο αντίτυπο υπο έκδοση εγχειριδίου από τις εκδόσεις Κάλλιπος

# Βιβλιογραφία

[1] “Object (Java Platform SE 8 ).” https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#equals-java.lang.Object- (accessed Oct. 15, 2021).

[2] O. O. Adekunle, Word Stemming/Hashing Algorithm: For Mail Classification. Place of publication not identified: LAP LAMBERT Academic Publishing, 2011.

[3] J.-P. Aumasson, W. Meier, R. Phan, and L. Henzen, The Hash Function BLAKE. Berlin Heidelberg: Springer-Verlag, 2014. doi: 10.1007/978-3-662-44757-4.

# Κεφάλαιο 12

**Σύνοψη**
*Στην ενότητα αυτή παρουσιάζονται κλάσεις της βιβλιοθήκης της Java που χρησιμοποιούνται συχνά κατά την ανάπτυξη εφαρμογών. Πιο συγκεκριμένα παρουσιάζονται οι κλάσεις String και StringBuilser για διαχείριση συμβολοσειρών, η κλάση ArrayList τα αντικείμενα της οποίας είναι στην ουσία εξελιγμένοι πίνακες, η κλάση LinkedList που αναπαριστά διασυνδεδεμένες λίστες, η κλάση Stack που υποστηρίζει δομές LIFO, η PriorityQueue που υποστηρίζει ουρές προτεραιότητας, οι κλάσεις HashSet και TreeSet που υποστηρίζουν διαχείριση συνόλων, οι κλάσεις HashMap και TreeMap που υποστηρίζουν απεικονίσεις, δηλαδή συσχέτιση ενός κλειδιού με δεδομένα, η κλάση BigDecimal που υποστηρίζει διαχείριση πραγματικών αριθμών αντιμετωπίζοντας τα προβλήματα που πηγάζουν από τον προσεγγιστικό χαρακτήρα τους και τέλος, μια ομάδα κλάσεων που διευκολύνουν τη διαχείριση ημερομηνιών.*

**Προαπαιτούμενη γνώση**
*Το διαδικαστικό μοντέλο προγραμματισμού με Java και η Ενσωμάτωση.*

**Λέξεις κλειδιά**
*Λίστες (List), Ουρές (Queue), Ουρές προτεραιότητας (Priority Queue), κατακερματισμός (hashing), BigDecimal, Απεικονίσεις (Map), Δένδρα (Tree), Σύνολα (Set), Αμεταβλητότητα (immutability).*

## 12 Αναγκαίες κλάσεις

Στην ενότητα αυτή παρουσιάζουμε βασικές κλάσεις της βιβλιοθήκης της Java που χρησιμοποιούνται συχνότατα και θα μας είναι χρήσιμες κατά την ανάπτυξη των ασκήσεων και άλλων εφαρμογών. Καταρχάς, η συζήτηση αφορά τις συμβολοσειρές, με την παρουσίαση των κλάσεων String και StringBuilder. Στη συνέχεια, εξετάζονται οι πιο βασικές συλλογές, δηλαδή κλάσεις κατάλληλες για την αποθήκευση αντικειμένων. Μετά παρουσιάζονται ορισμένες απεικονίσεις, δηλαδή κλάσεις με τις οποίες μπορούμε να συσχετίζουμε κλειδιά με δεδομένα, ακολουθούμενες από την κλάση BigDecimal για αναπαράσταση και διαχείριση πραγματικών τιμών και τέλος συζητάμε βασικές κλάσεις διαχείρισης ημερομηνιών.

Να σημειωθεί ότι δεν γίνεται εξαντλητική παρουσίαση των κλάσεων. Παρουσιάζεται η βασική λειτουργικότητά τους και οι πιο συχνά χρησιμοποιούμενες μέθοδοι και μόνον εφόσον αυτές είναι δυνατόν να κατανοηθούν με βάση τη γνώση που έχει παρουσιαστεί μέχρι εδώ σε αυτό το βιβλίο.

### 12.1 Η κλάση String

Κάποια στοιχεία από την κλάση String έχουν ήδη παρουσιαστεί. Εδώ θα παρουσιάσουμε μια σειρά από συναρτήσεις στιγμιότυπου και τους βασικούς δημιουργούς της κλάσης.

```
String s1 = new String("John");
String s2 = "John";
char[] p = {'M', 'a', 'r', 'y'};
String s3 = new String(p);
```

***Κώδικας 12.1*** *Οι βασικοί δημιουργοί της κλάσης String*

Στον κώδικα 12.1 βλέπουμε παραδείγματα των βασικών δημιουργών της κλάσης. Οι δύο πρώτοι δημιουργοί έχουν ήδη παρουσιαστεί (ενότητα 3.8). Στην τελευταία γραμμή βλέπουμε έναν δημιουργό που λαμβάνει ως παράμετρο έναν πίνακα από χαρακτήρες και συνθέτει ένα String.

**Συναρτήσεις**

```
char charAt(int index)
```

επιστρέφει τον χαρακτήρα που βρίσκεται στη θέση index αυτού του String, δηλαδή του String που την καλεί.

```
int compareTo(String anotherString)
```

Επιστρέφει 0 αν αυτό το String είναι ίσο με το anotherString, αρνητικό ακέραιο αν αυτό το String είναι λεξικογραφικά μικρότερο από το anotherString και θετικό ακέραιο, αν αυτό το String είναι μεγαλύτερο. Για παράδειγμα, η πρόταση

```
System.out.println((s1 + " Black").compareTo(s2));
```

θα επιστρέψει θετικό ακέραιο. Πιο συγκεκριμένα, η αξιολόγηση της έκφρασης μέσα στην println γίνεται ως εξής: Πρώτα αξιολογείται η σύνδεση s1+" Black" που αποκτά προτεραιότητα καθώς περικλείεται σε παρένθεση. Το αποτέλεσμα αυτής της σύνδεσης είναι το String John Black. Το String αυτό συγκρινόμενο με την παράμετρο s2 που έχει περιεχόμενο John θεωρείται λεξικογραφικά μεγαλύτερο.

```
int compareToIgnoreCase(String anotherString)
```

Συγκρίνει αυτό το String με την παράμετρό της χωρίς να λαμβάνει υπόψη τυχόν διαφορές μεταξύ πεζών και κεφαλαίων. Για παράδειγμα, η κλήση

```
s1.compareToIgnoreCase("john");
```

επιστρέφει 0, παρότι το s1 έχει ως περιεχόμενο το John.

```
boolean endsWith(String suffix)
```

Επιστρέφει true εφόσον η παράμετρος suffix είναι το τελευταίο τμήμα αυτού του String. Αντίστοιχα, λειτουργεί και η startsWith.

```
boolean equalsIgnoreCase(String anotherString)
```

Επιστρέφει true εφόσον αυτό το String είναι ίσο με την παράμετρό της χωρίς να λαμβάνονται υπόψη διαφορές μεταξύ πεζών και κεφαλαίων. Η συνάρτηση μοιάζει πολύ με την compareToIgnoreCase με τη διαφορά ότι είναι boolean, ενώ η compareToIgnoreCase επιστρέφει int. Βεβαίως, όπως όλες οι κλάσεις της Java, η κλάση String υποστηρίζει μέθοδο equals.

```
int indexOf(String str)
```

Επιστρέφει τη θέση της str μέσα σε αυτό το String. Αν η str δεν βρεθεί, επιστρέφει -1.
Η συνάρτηση lastIndexOf(String str) επιστρέφει την τελευταία θέση της str μέσα σε αυτό το String. Αν η str δεν βρεθεί, επιστρέφει -1.

```
int indexOf(String str, int fromIndex)
```

Επιστρέφει τη θέση της str μέσα σε αυτό το String εφόσον αυτό βρεθεί από τη θέση fromIndex και μετά, διαφορετικά, επιστρέφει -1.

```
int length()
```

Επιστρέφει το μήκος αυτού του String.

```
boolean isEmpty()
```

Επιστρέφει true μόνον εφόσον το μήκος του String είναι ίσο με το 0.

```
boolean isBlank()
```

Επιστρέφει true αν το μήκος του String είναι 0 ή αν το String περιέχει μόνο λευκούς χαρακτήρες (whitespace characters). Σημειώστε πως λευκοί χαρακτήρες είναι όσοι χαρακτήρες ή σειρές χαρακτήρων αντιπροσωπεύουν τυπογραφικά οριζόντιο ή κάθετο διάστημα. Για παράδειγμα, ο κώδικας

```
System.out.println(s.length()+"                "+s.isEmpty()+"
"+s.isBlank());
```

θα τυπώσει

```
0 true true
```